

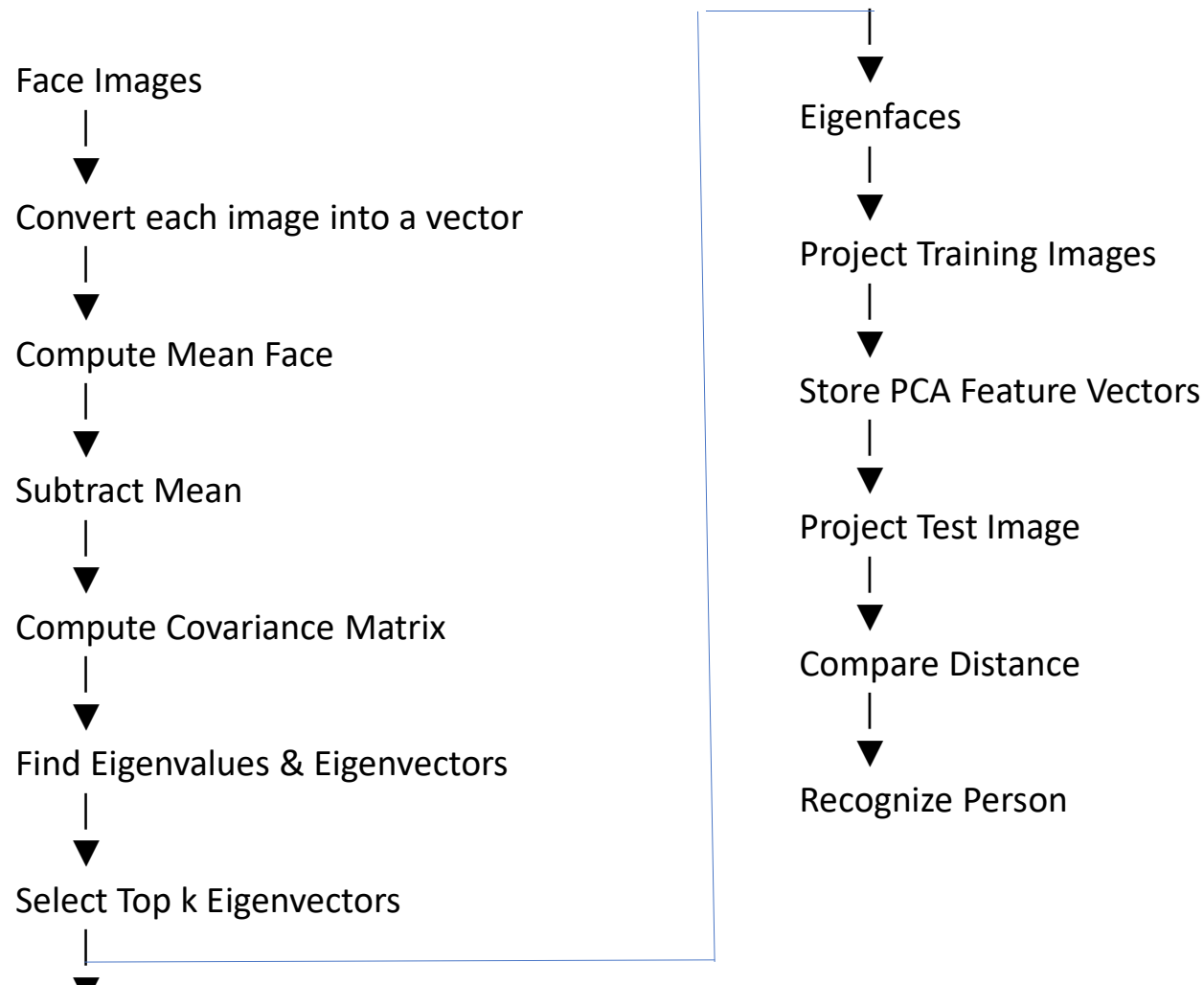
Recall : PCA (We studied in AI&NN)

- **Principal Component Analysis (PCA)** is one of the earliest and most widely used dimensionality reduction techniques in **facial biometric systems**.
- It is popularly known as the **Eigenfaces** method, proposed by Turk and Pentland (1991).
- The main idea is that instead of storing every pixel of a face image, PCA extracts only the most important information (principal components), thereby reducing computation while preserving facial characteristics.

Why PCA is Required in Facial Biometrics

- Consider a grayscale face image of size **100 × 100 pixels**.
- Total pixels = ? $100 \times 100 = \mathbf{10,000}$ features
- Every image becomes a vector of length **10,000**
- Suppose we have 500 face images
- Each image has 10,000 pixel values
- The dataset size becomes $500 \times 10,000$.
- Training directly on these 10,000-dimensional vectors is computationally expensive , memory intensive , highly redundant because neighboring pixels are strongly correlated.
- PCA solves this problem.

Overall PCA Workflow in Face Recognition



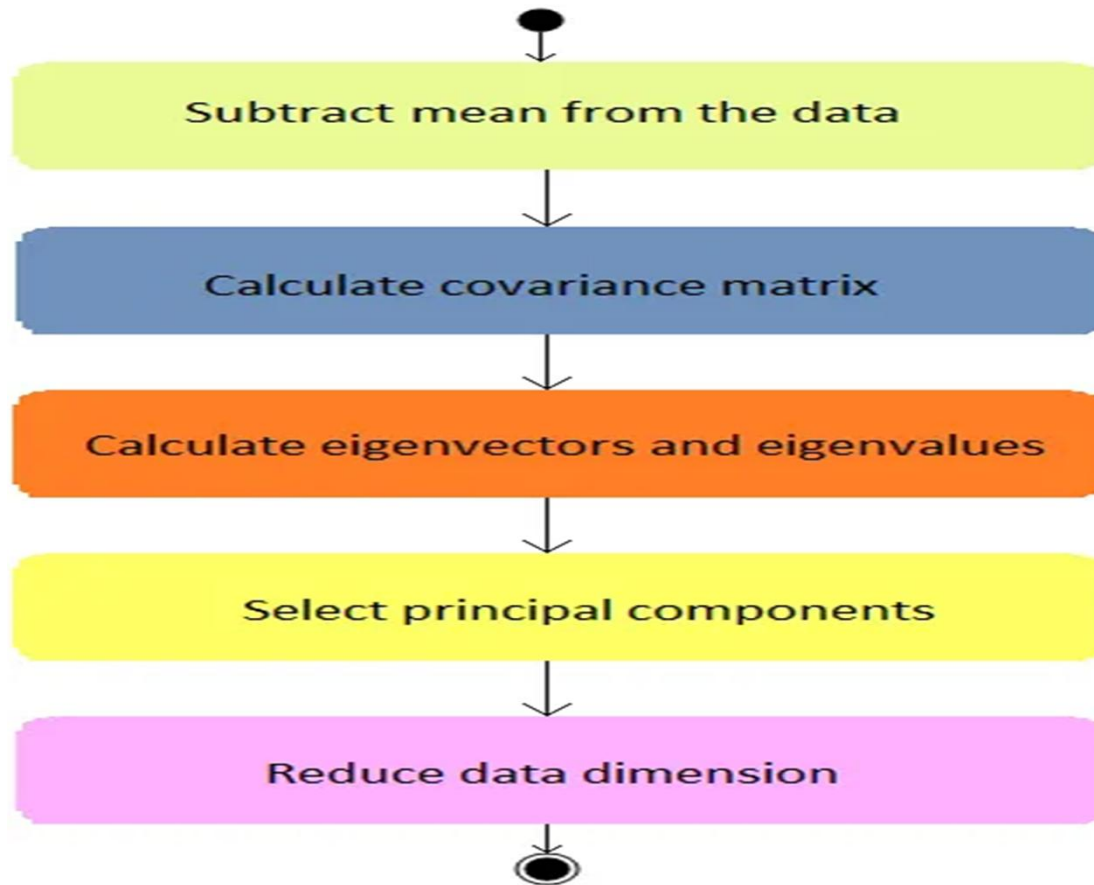


Principal Component Analysis (PCA)

- Eigenvectors and eigenvalues are the linear algebra concepts that are used to compute the covariance matrix to determine the principal component of the data.
- Why PCA is popular?
- The reason why it chooses the direction of bigger variance because it could significantly compress the dimension without losing a lot of information.



Steps In PCA



Assume we have 4 training images.

- Each image is only **2 × 2 pixels** (for simplicity).

Image 1	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	Image 3	$\begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$
Image 2	$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$	Image 4	$\begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix}$

1,2,3,4 ... 7 represent the pixel intensity values of an image.

In a real face image, every pixel stores a brightness (grayscale) value.

0 → Black

255 → White

128 → Gray

For simplicity, instead of using values from 0–255, I used small numbers (1–7) so that the PCA calculations are easy to understand.

Step 1 Convert Images into Vectors

Flatten row-wise.

Image 1

[1, 2, 3, 4]

Image 2

[2, 3, 4, 5]

Image 3

[3, 4, 5, 6]

Image 4

[4, 5, 6, 7]

Training matrix

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 3 & 4 & 5 & 6 \\ 4 & 5 & 6 & 7 \end{bmatrix}$$

Each row represents one face.

Step 2 Compute Mean Face (Mean of each column)

Column 1	$(1 + 2 + 3 + 4)/4 = 2.5$
Column 2	3.5
Column 3	4.5
Column 4	5.5
Mean face	$\mu = [2.5, 3.5, 4.5, 5.5]$

Mean face is

Mean

$$\Psi = \begin{bmatrix} 2.5 \\ 3.5 \\ 4.5 \\ 5.5 \end{bmatrix}$$

Step 3: Normalize
(Subtract the mean)

$$\Phi_1 = \begin{bmatrix} -1.5 \\ -1.5 \\ -1.5 \\ -1.5 \end{bmatrix}$$

$$\Phi_2 = \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{bmatrix}$$

$$\Phi_3 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

$$\Phi_4 = \begin{bmatrix} 1.5 \\ 1.5 \\ 1.5 \\ 1.5 \end{bmatrix}$$

Form Matrix A

Arrange normalized vectors as columns.

$$A = \begin{bmatrix} -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \end{bmatrix}$$

Step 4 Covariance Matrix

Covariance measures how features vary together.

PCA computes

$$C = \frac{1}{n-1} X^T X$$

Instead of looking at individual pixels, covariance reveals relationships between them.

Step 5 Eigenvalues and Eigenvectors

- Suppose the covariance matrix gives
- Eigenvalues
- [18, 2, 0.4, 0.1]
- Largest eigenvalue 18, Second is 2
- Since the first two eigenvalues explain almost all the variance,
- we keep only Eigenface 1 , Eigenface 2
- Discard Eigenface 3 and Eigenface 4

What is an Eigenface?

- An eigenface is **not a real face**.
- It is a mathematical pattern representing a major direction of variation in facial appearance.

Suppose

Eigenface 1

$$v_1 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Suppose the second eigenvector is

$$v_2 = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.5 \\ -0.5 \end{bmatrix}$$

These two vectors form our **Eigenface Space**.

Step 6: Compute the Weights of Every Training Face

- Formula

$$w = v^T \Phi$$

Each face gets one weight for each Eigenface.

Face 1

Normalized

$$(-1.5, -1.5, -1.5, -1.5)$$

Weight on Eigenface 1

$$\begin{aligned}w_1 &= 0.5(-1.5) + 0.5(-1.5) + 0.5(-1.5) + 0.5(-1.5) \\&= -3\end{aligned}$$

Weight on Eigenface 2

$$\begin{aligned}0.5(-1.5) + (-0.5)(-1.5) + 0.5(-1.5) + (-0.5)(-1.5) \\= -0.75 + 0.75 - 0.75 + 0.75 = 0\end{aligned}$$

Therefore

Face 1

$$(-3, 0)$$

Face 2

Normalized

$$(-0.5, -0.5, -0.5, -0.5)$$

Weight 1

$$0.5(-0.5) \times 4 = -1$$

Weight 2

$$0$$

Face 2

$$(-1, 0)$$

Face 3

Normalized

$$(0.5, 0.5, 0.5, 0.5)$$

Weight 1

$$1$$

Weight 2

$$0$$

Face 3

$$(1, 0)$$

Face 4

Normalized

$$(1.5, 1.5, 1.5, 1.5)$$

Weight 1

$$3$$

Weight 2

$$0$$

Face 4

$$(3, 0)$$

Final Training Database

Person	Weight Vector
Face 1	$(-3,0)$
Face 2	$(-1,0)$
Face 3	$(1,0)$
Face 4	$(3,0)$

Testing : Recognition Phase

Now suppose a new image arrives.

New face

$$\begin{bmatrix} 2.2 & 3.2 \\ 4.2 & 5.2 \end{bmatrix}$$

Convert to vector

$$\Gamma = \begin{bmatrix} 2.2 \\ 3.2 \\ 4.2 \\ 5.2 \end{bmatrix}$$

Testing : Recognition Phase

Normalize

Subtract mean

$$\Phi = \begin{bmatrix} -0.3 \\ -0.3 \\ -0.3 \\ -0.3 \end{bmatrix}$$

Compute Weights

Eigenface 1

$$w_1 = 0.5(-0.3) \times 4 = -0.6$$

Eigenface 2

$$0$$

Weight vector

$$(-0.6, 0)$$

Euclidean Distance

Distance to Face 1

$$\sqrt{(-0.6 + 3)^2} = 2.4$$

Distance to Face 2

$$\sqrt{(-0.6 + 1)^2} = 0.4$$

Distance to Face 3

$$\sqrt{(-0.6 - 1)^2} = 1.6$$

Distance to Face 4

$$\sqrt{(-0.6 - 3)^2} = 3.6$$

Decision

Smallest distance 0.4.
Therefore

- **New face is recognized as Face 2.**